

Отчёт к ЛР №3. ИГИ. Стандартные типы данных, коллекции, функции, модули.

Луканский Дмитрий, гр.453502

Вариант 14

Задание 1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд.

Задать точность вычислений ϵ .

Предусмотреть максимальное количество итераций, равное 500.

Вывести количество членов ряда, необходимых для достижения указанной точности вычислений. Результат получить в виде:

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество

просуммированных членов ряда, $\text{Math } F(x)$ – значение функции, вычисленное с помощью модуля `math`.

Для функции $\ln(1-x)$

использовался ряд

Тейлора.

Для выполнения задания

была создана генератор

частичных сумм ряда

`row_generator`,

функция, вычисляющая

сумму при помощи

модуля `math`

`calculate_row_sum()`, и

генератор таблицы

`table_generator()`, который

передаётся в функцию

модуля `UI` `show_table`.

```
def row_generator(x):
    """
    Generator of series terms for computing ln(1 - x).

    Yields tuples (n, partial_sum), where partial_sum is the partial sum
    of the series -x - x^2/2 - x^3/3 - ... for the given x.
    Generation continues up to n = 500 (loop limit).

    Parameters:
    | x (float): Argument for which the series is computed. |x| < 1.

    Yields:
    | tuple: (term index n, partial sum up to n).
    """
    cur_n = 1
    cur_num = x
    cur_sum = -cur_num

    while cur_n < 500:
        yield (cur_n, cur_sum)
        cur_n += 1
        cur_num = (cur_num * x) * ((cur_n - 1) / cur_n)
        cur_sum += -cur_num

def calculate_row_sum(x):
    """
    Computes the exact value of ln(1 - x) using math.log.

    Parameters:
    | x (float): Argument for the logarithm. |x| < 1.

    Returns:
    | float: The value of ln(1 - x).
    """
    return math.log(1 - x)

def table_generator(x, eps, row_sum):
    """
    Generator of table rows showing convergence of the series.

    For each term from row_generator, calculates the absolute error
    relative to the exact value row_sum. Stops when error <= eps.

    Parameters:
    | x (float): Argument of the series.
    | eps (float): Required precision.
    | row_sum (float): Exact value of ln(1 - x).

    Yields:
    | tuple: (x, n, partial_sum, exact_sum, error).
    """
    for cur_n, cur_val in row_generator(x):
        cur_precision = abs(cur_val - row_sum)
        yield(x, cur_n, cur_val, row_sum, cur_precision)
        if cur_precision <= eps:
            break
```

```
def main():
    """
    Main function of the program.

    Prompts the user for x ( $|x| < 1$ ) and precision eps.
    Computes the exact value of  $\ln(1 - x)$  and displays a table
    showing the convergence of partial sums to that value.
    """
    x = ui.read_float("Enter x for computing  $\ln(1 - x)$ ,  $|x| < 1$ \n", strict_min = -1.0, strict_max = 1.0)
    eps = ui.read_float("Enter precision of computing row eps:\n", min = 1e-9, max = 1)

    row_sum = calculate_row_sum(x)
    ui.show_table([table_generator(x, eps, row_sum), ["x", "n", "F(x)", "sum", "eps"]])
```

Задание 2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел. Организовать цикл, принимающий целые числа с клавиатуры и подсчитывающий количество отрицательных чисел. Окончание цикла – ввод отрицательного числа

```
import ui

def main():
    """
    Main function that repeatedly subtracts user-provided integers from 10000.

    Prompts the user to enter integers one by one. Each entered integer is
    subtracted from the current result, which starts at 10000. After x becoming
    less than 0 loop stops and the current result is printed
    """
    cur_res = 10000
    while cur_res >= 0:
        cur_num = ui.read_int("Enter an integer ")
        cur_res -= cur_num

    print(f"Result is {cur_res}")
```

Задание 3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры. Посчитать число пробелов и запятых в строке с помощью функции.

```
'''
Laboratory work number: 3. Standard data types, collections, functions, modules.
Version: 1.0.
Developer: Lukanski Dzmitry Mikalaevich.
Date: 25.03.2026.

This module contains task calculating number of " " and " ,"
'''

import ui

def main():
    """
    Analyzes a user-provided string by counting spaces and commas.

    Prompts the user to enter a string, then prints the number of space characters
    (' ') and comma characters (',') found in the string using the count() method.
    """
    str = ui.read_str("Enter a string to analyse:\n")
    print(f"Number of \\' \' is {str.count(" ")}")
    print(f"Number of \\' ,\' is {str.count(",")}")
```

Задание 4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы: «So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.»

Реализованы функции: подсчёт количества слов длины меньше 7, нахождение самого короткого слова, вывод слов в порядке убывания длины.

```
import ui

def calc_words_with_len_less_than_7(words):
    """
    Counts how many words have a length less than 7.

    Parameters:
    | words (list of str): A list of words (strings).

    Returns:
    | int: The number of words whose length is less than 7.
    """
    lens = [len(word) for word in words]
    return sum(1 for len in lens if len < 7)

def find_min_len_word_ends_with_a(words):
    """
    Finds the shortest word that ends with the letter 'a'.

    If multiple words have the same minimal length, the first encountered is returned.
    If no word ends with 'a', returns an empty string.

    Parameters:
    | words (list of str): A list of words (strings).

    Returns:
    | str: The shortest word ending with 'a', or an empty string if none found.
    """
    lens = [len(word) for word in words]
    min_len = max(lens) + 1
    min_len_word = ""
    for word in words:
        if word.endswith('a') and len(word) < min_len:
            min_len = len(word)
            min_len_word = word
    return min_len_word

def main():
    """
    Main function that reads a string from the user, splits it into words,
    removes commas from each word, and then:
    1. Prints the number of words shorter than 7 characters.
    2. Prints the shortest word ending with 'a'.
    3. Prints the list of words sorted by length in descending order.
    """
    str = ui.read_str("Enter a string to analyse:\n")
    words = str.split(" ")
    for i in range(len(words)):
        words[i] = "".join(character for character in words[i] if character != ',')

    print(f"Number of words with length < 7: {calc_words_with_len_less_than_7(words)}")

    print(f"Word with minimum length from ending with 'a' is {find_min_len_word_ends_with_a(words)}")

    print(sorted(words, key = lambda word: len(word), reverse=True))
```

Задание 5. соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции: 1) ввод элементов списка пользователем; 2) проверка корректности вводимых данных; 3) реализация основного задания с выводом результатов; 4) вывод списка на экран. Найти минимальный отрицательный элемент и сумму элементов между первым и вторым отрицательным

```
import list_initialization
import ui

def find_min_neg_ind(list):
    """
    Finds the index of the minimum negative element in the list.

    Uses min() with a key to locate the smallest value in the list.
    If that smallest value is negative, returns its index.
    Otherwise, returns -1 indicating no negative elements exist.

    Parameters:
        list (list of int/float): The list to search.

    Returns:
        int: Index of the smallest negative element, or -1 if no negatives.
    """
    ind, val = min(enumerate(list), key = lambda x : x[1])
    if val < 0:
        return ind
    else:
        return -1

def find_sum(list):
    """
    Computes the sum of elements between the first and second negative element.

    Finds the indices of the first two negative numbers in the list.
    Then sums all elements strictly between those indices (excluding the negatives).
    If fewer than two negative elements exist, returns 0.

    Parameters:
        list (list of int/float): The input list.

    Returns:
        int/float: Sum of elements between the first and second negative element.
    """
    ind1 = -1
    ind2 = -1
    for i in range(len(list)):
        if (ind1 != -1 and list[i] < 0):
            ind2 = i
            break
        if (list[i] < 0):
            ind1 = i

    sum = 0
    for i in range(ind1 + 1, ind2):
        sum += list[i]

    return sum

def main():
    """
    Main entry point of the program.

    Initializes a list using list_initialization.initialize_list()
    """
```

```
def main():
    """
    Main entry point of the program.

    Initializes a list using list_initialization.initialize_list(),
    prints the list, finds the index of the minimum negative element,
    and computes the sum of elements between the first two negative elements.
    Displays the results or appropriate messages if conditions are not met.
    """
    list = list_initialization.initialize_list()

    min_neg_ind = find_min_neg_ind(list)
    sum_bt看_neg_1_ans_2 = find_sum(list)

    print(list)

    if min_neg_ind != -1:
        print(f"Minimum negative element index is {min_neg_ind}")
    else:
        print("There are no negative elements in the list")
    print(f"Sum of elements between 1 and 2 negative elements is {sum_bt看_neg_1_ans_2}")
```